

IN2010-innleveringsoppgave1

Oppgave 2

2a)

push_back: node som skal legges til bak

input: node plassert i listen

output:

leggTil(x)

siste.neste = ny

ny.forrige = siste

siste = ny

teller++

push_front

input: ny node som skal legges til foran

output: noden plassert i listen

leggTilStart()

Node ny <- new Node

ny.neste <- start

start <- ny

if (storrelse = 1) then

siste <- ny

teller++

push_middle

input: ny node som skal legges til i midten

output: noden plassert i listen

leggTilMiddle()

Node ny <- new Node

Node tmp = start

for i <- 0 to storrelse()/2

tmp = tmp.neste

ny.neste = tmp.neste

tmp.neste = ny

teller++

get

Input: ønsket plassering på node

Output: Henter ut ønsket node

hent(pos)

for i <- 0 to pos

node = node.next

return node.data

2c)
push_back
O(1)
push_middle
O(n)
push_front
O(1)

totalt vil kjøretiden være $O(n)$

2d)
det er viktig at vi fjerner begrensningen på N i forrige deloppgave for det vil tillate oss å gjøre om alle konstanter til "1". Dette lar oss regne mye grovere, samt at vi får bevart hovedtrenden i hvordan kjøretiden vokser når input blir større. når vi fjerner begrensningen vil N ikke være en konstant og heller bli en ukjent faktor som kan være uendelig stor.

Oppgave 3

input: første linje er verdien til plassen til katten. første tall i hver linje er foreldre, resten er barnet.

output: veien katten går fra sin node til roten

lesInn()

```
x <- scanner.nextline
data[] <- scanner.nextline.split(" ")
rot <- data[0]
map <- new HashMap
teller <- 0
for i <- 0: i < data.length: i++
    map.put(data[teller + 1], data[0])
    teller++
```

```
while !scanner.equals("")
    data <- scanner.nextline.split(" ")
    Y <- 0
    for i <- 0: i < data.length: i++
        map.put(data[i+1], data[0])
        Y++
```

```
sout(x)
while x != rot
    x = map.get(x)
    sout(x)
```

Oppgave 4

a) Psudokode:

Input(Array): putter inn et sortert Array a
output: Lage en ny array slik at hvis man skulle lage et binært tre
ville den vært balansert

nyArray(); lager ny array med samme størrelse som input array.
binarySearch(array): binært søk gjennom input array og setter det i ny array

binarySearch(array):

midten = oldArray.size()/2
nyArray.add(oldArray[midten] gir data fra midten

hentVenstre(l, m); Sett parameter l og m. Legg sammen og del på 2 så får du
mid. Legg oldArray[mid] inn i ny array.
hentVenstre(l, mid)
hentHoyre(mid, m)

hentHoyre(m,h); Sett parameter l og m. Legg sammen og del på 2 så får du
mid. Legg oldArray[mid] inn i ny array.
hentVenstre(l, mid)
hentHoyre(mid, m)

forskjellen på hentHøyre og venstre er de har forskjellige parametre.

b) Psudokode

Input(max); max bestemmer størrelsen på heap
output() balansert heap

leggTil(x); x er verdien du skal legge til heap-arrayet;

size += 1;

sjekker samtidig om forelder noden er større. hvis den er det

bytt(x,y); x og y er index og du skal bytte x med y for å
balansere heapen.